

找出顺序

- 时间限制：10 分钟
- 环境：一块 GPU (≈ 16 GB VRAM)，无互联网
- 解决方案大小： `solution.ipynb` ≤ 1 MB
- 存储空间：5 GB

问题

给定由两名参与者 说话者 A 和 说话者 B 进行的英语口语对话。每段对话被分割为若干说话轮次，每个轮次仅包含一名说话者的语音。每个轮次均存储为单独的 `.wav` 音频文件，因此一段完整的对话由一组 `.wav` 文件表示，每个轮次对应一个文件。

遗憾的是，这些轮次已被随机打乱，因此对话不再合乎逻辑。在文件名 `chunk_{k}.wav` 中， k 表示打乱后集合中的第 k 个片段，而不是原始对话中的第 k 个轮次。

!! 你的任务是重建对话原始的时间顺序。



数据集

每段对话包含 n 个音频文件，文件名为 `chunk_0.wav`、`chunk_1.wav`、 $\cdots$ 、`chunk_{n-1}.wav`。这些片段分别对应单独的轮次。文件名仅对应打乱后的顺序，并不表明片段在原始对话中的位置。每段对话包含 7-20 个片段，单声道，44.1 kHz（你可以重采样）。

`prefix.json` 包含每段对话前两个片段的文件名索引。这确定了对话真正的开头，并消除了正向或反向读取对话所产生的歧义。

例如：11: [7, 12] 表示对话 11 的第一个和第二个轮次分别是 `chunk_7.wav` 和 `chunk_12.wav`。

你将获得的内容

你会收到两个格式完全相同的文件夹：

文件夹	对话数	answers.json?	用途
dataset/train/	1,288	✔ 包含	训练／微调你的模型
dataset/test_public/	100	✔ 包含	运行你的流程并在本地自行评分

评分时，你的 `dataset/test_public/` 文件夹会被透明地替换为 `hidden evaluation set`（`test_leaderboard_a` 用于公开排行榜，`test_leaderboard_b` 用于最终排行榜）——它们的大小和格式与 `dataset/test_public/` 相同，但不包含 `answers.json`。

你的 notebook 会在该数据上再次执行，其生成的 `answers.json` 文件将用于评分。留出的测试对话与 `train` 来自相同分布，因此你本地的 `test_public` 分数能够如实预示最终表现。

目录结构

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

输出

对于每段对话，确定其音频片段原始的时间顺序。你的预测应为 $\{0, 1, \dots, n-1\}$ 的一个排列 P ，其中 $P[i]$ 是 `chunk_i.wav` 的预测时间位置（0 = 第一个）。

你的输出文件 `answers.json` 应将每个对话 ID 映射到其预测排列：

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

示例

一段对话有 3 个被打乱的片段 `chunk_0`, `chunk_1`, `chunk_2`：

打乱后的片段	口语内容	真实位置（排名）
<code>chunk_0.wav</code>	<i>"No worries — I'll send you the notes afterwards."</i>	2（最后一个）
<code>chunk_1.wav</code>	<i>"Hey, are you coming to the three o'clock meeting?"</i>	0（第一个）
<code>chunk_2.wav</code>	<i>"I can't — I've got a dentist appointment then."</i>	1

真实顺序为 **chunk_1** → **chunk_2** → **chunk_0**，因此 $P = [2, 0, 1]$ ，且 `prefix.json` 保存 `[1, 2]`。

⚠ **P 必须是一个真正的排列**：长度为 n 、从 0 开始索引，且每个值恰好出现一次。存在重复值、缺失值或越界项（例如从 1 开始索引）时，该对话得分为 0；文件中缺少某段对话时同样如此。格式错误或非 JSON 文件将被拒绝。

评分

本任务的评分指标为**成对顺序准确率**。它会检查每一对片段并询问：*两者中的哪一个应当在前*？如果你的预测与真实答案给出的结果相同，则该片段对正确。对于包含 n 个片段的对话，共有

$$M = n(n - 1)/2$$

个片段对；令 I 表示逆序数，即与真实答案顺序不同的片段对数量：

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

i 最终分数是该数据划分中所有 对话的逐对话分数的平均值。

允许使用的模型

在训练和评估期间，你只能使用以下预训练模型来解决本任务。所有这些模型都已下载并可在环境中使用。你可以在 `baseline notebook solution.ipynb` 中查看它们的使用示例。请注意，你不能使用任何其他模型，并且你的程序无法访问互联网。

- **语音表征**：**wav2vec 2.0**。也可以将 **Whisper encoder** 用作特征提取器。 [wav2vec 模型卡](#)
- **自动语音识别 (ASR)**：**OpenAI Whisper**（任意大小）。 [Whisper 模型卡](#)
- **语言模型**：**Qwen2.5-0.5B**，既可以零样本使用，也可以在提供的 `train` 数据划分上进行微调。 [Qwen 模型卡](#) 请注意，10 分钟的限制必须涵盖你在评分时进行的任何训练或微调，以及在评估集上的推理。

如何提交

- 打开 `solution.ipynb` 并运行所有单元格。确认它会在工作目录中写入 `answers.json`，并为 `dataset/test_public/` 中的每段对话（100 段对话）提供一个排列。评分时，notebook 会在隐藏测试集上重新运行，并对它在那里生成的 `answers.json` 进行评分。
- 如果愿意，你可以改进解决方案，也可以不改进；仅使用 baseline 即可验证整个流程。
- 打开 JupyterLab 左侧边栏中的 Git 选项卡。
- **Stage** `solution.ipynb`（其旁边的 + 图标）。
- 输入提交消息，然后点击 **Commit**。
- 点击带向上箭头的云形图标以推送。

- 返回此竞赛页面并点击 **Submit** 。

只提交一个文件，文件名为 `solution.ipynb` 。